UNIT 3 · DEVOPS, CI/CD, CONTAINERIZATION & APPLICATION DEPLOYMENT

Enterprise Student Record & Academic Management Platform

DevOps Foundation, Version Control, Continuous Integration & Container Orchestration

Technical Project Report — theory, implementation and verification

Class	SY-AIDS
Topic	Enterprise Student Record & Academic Management Platform: DevOps, Version Control, CI & Container Orchestration
Environment	Python · Flask · Pytest · Jenkins · Docker · Minikube
Repository	https://github.com/amaan974/DEVOPS-1 (folder `Unit-3`)
Date	17 October 2026

Contents

Part A — DevOps foundations applied to this project 2

1. Executive summary and engineering context 2

2. The business need for DevOps 3

3. The end-to-end DevOps lifecycle 4

4. Version control and Git workflows 5

5. Continuous integration with Jenkins 6

6. Containerization and Docker 7

7. Kubernetes and Minikube orchestration 8

8. The integrated DevOps ecosystem 9

Part B — Implementation and verification 12

9. Application design 12

10. Setup steps 13

11. Verification results and evidence 14

12. Challenges and resolutions 17

13. Reflections and next steps 18

14. Conclusion 18

References 18

Appendix. Key command output 19

Part A — DevOps foundations applied to this project

Chapters 1–8 follow the structure of the course's theoretical reference (Unit 3, Document 1). Each concept is explained in terms of what it meant for building and running the Enterprise Student Record & Academic Management Platform.

1. Executive summary and engineering context

This report documents how the Enterprise Student Record & Academic Management Platform — a Flask service for student records, course registration, marks and GPA — was taken from source code to a self-healing Kubernetes deployment with Git, Jenkins, Docker and Minikube. Earlier units covered requirements, backlogs and sprint planning; Unit 3 is where those plans become an engineering system in which every change is versioned, built, tested, packaged and deployed by automation instead of by hand.

DevOps is treated here as a way of working rather than a set of tools. The delivery process itself is written as code — a `Jenkinsfile`, a `Dockerfile` and Kubernetes YAML — and lives in the same repository as the application, so a change to how the software is shipped is reviewed and versioned exactly like a change to the software.

> **CORE OBJECTIVE**
>
> DevOps joins development and operations through **Culture, Automation, Measurement and Sharing (CAMS)**.
>
> The measurable goal is a short, safe path from commit to running software: deploy often, fail rarely, and recover quickly (a low mean time to recovery, MTTR) when something does break.

Project at a glance

Application	Enterprise Student Record & Academic Management Platform (Flask 3.0, Python 3.11)
Based on	The trainer's Enterprise Smart Library Management Platform case study (Unit 3, Document 2)
Entities	Students, Faculty, Courses, Enrollments, Grades
Key business rule	At most 18 credits per student per semester; plus prerequisites, timetable clashes and course capacity
Quality gates	flake8, 107 pytest tests, coverage ≥ 85%, container smoke test
Release artifact	Docker image `student-academic-platform:v1.0.<BUILD_NUMBER>`
Runtime	Minikube: Deployment `student-deployment` (3 replicas), Service `student-service` on NodePort 30090

107
automated tests passed

99.7%
code coverage (gate 85%)

9 s
to replace a deleted pod

0
failed requests during a rolling update

2. The business need for DevOps

2.1 Why traditional delivery struggles

In a siloed organisation developers are rewarded for shipping features and operations staff for keeping systems stable, so every release becomes a negotiation between two teams with opposite incentives — the so-called *wall of confusion*. Releases become rare and large, and problems surface late, when they are most expensive to fix.

- **Late integration.** Work developed in isolation for weeks is merged at the end, so conflicts and regressions all arrive at once (“integration hell”).
- **Environment drift.** The developer's laptop, the test server and production run different packages and library versions, so code that passes in one place fails in another — the “works on my machine” problem.
- **Manual releases.** Deployments follow written runbooks, copy files by hand and edit configuration on live servers; every manual step is a chance for error.
- **Slow, risky recovery.** Undoing a bad release means rebuilding servers or restoring databases, so outages last hours.
- **Blame over learning.** Separate teams look for whose fault an incident was rather than how to prevent the next one.

The academic platform would face the same risks without automation. A change to the 18-credit rule tested on one laptop only could, for example, block every student's enrolment on registration day.

2.2 Waterfall, Agile and DevOps compared

Agile shortened the loop between users and developers, but it usually stopped at “working software on a developer's machine”. DevOps extends the same loop through building, deploying and operating the system.

Dimension	Traditional (Waterfall)	Agile	DevOps (this project)
Release cadence	One large release per phase, months apart	An increment every 2–4 week sprint	Any green commit can ship; each Jenkins build produces a release candidate
Team structure	Separate development, QA and operations	Cross-functional development and QA	One team owns build, test, deployment and operation
Path to production	Manual runbook	Often manual at the end of a sprint	Automated pipeline defined in the `Jenkinsfile`
Environments	Drift between development, test and production	Staging still differs from production	The same container image runs everywhere
Testing	After development, mostly manual	Inside the sprint, partly automated	On every commit; a failing test or coverage < 85% stops the build
Feedback	Post-release reviews	Sprint review and retrospective	Health probes, logs and the pipeline's own reports
Cost of a bad change	High; rollback is a project	Medium	Low; `kubectl rollout undo` restores the previous version

3. The end-to-end DevOps lifecycle

The lifecycle is a loop, not a line: what is learnt while operating the system feeds the next round of planning. Figure 1 maps each of the eight phases to what it concretely means for this project.

Every phase of the DevOps loop maps to a concrete step in this project

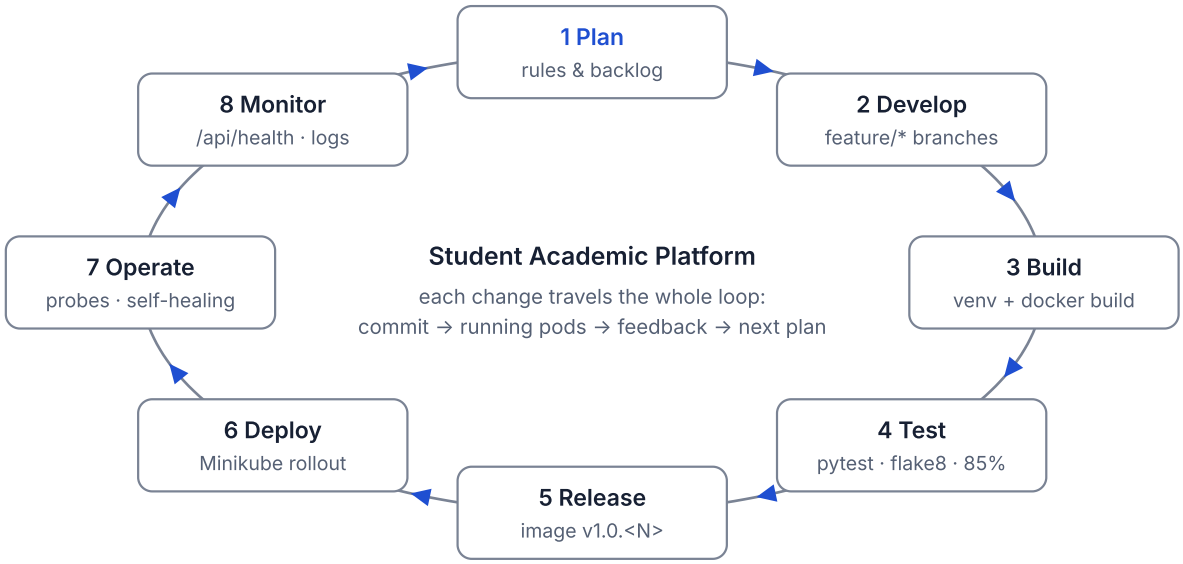


Figure 1. The eight lifecycle phases and how each is realised in the Student Academic Platform

3.1 Each phase in this project

- **Plan.** The case study's requirements were turned into explicit, testable rules before coding: the 18-credit cap, prerequisites, timetable clashes, course capacity and a 10-point grading scale.
- **Develop.** Code is written on short-lived `feature/*` branches, one per topic; the flake8 rules in `setup.cfg` apply the same style to every contributor.
- **Build.** A fresh virtual environment is created from pinned requirements, and Docker builds an image with the dependencies in their own cached layer.
- **Test.** flake8 and 107 pytest unit and HTTP-integration tests run on every build; coverage below 85% fails it.
- **Release.** The image is tagged with the Jenkins build number (`v1.0.<N>`) and labelled with the Git commit, so any running container can be traced back to its source.
- **Deploy.** Kubernetes replaces pods one at a time (`maxSurge: 1`, `maxUnavailable: 0`) and only sends traffic to a pod once its readiness probe passes.
- **Operate.** A liveness probe restarts a hung container, and the Deployment recreates any pod that crashes or is deleted.
- **Monitor.** `/api/health` reports status, version and the serving pod; Gunicorn's access log goes to standard output for `kubectl logs`. Metrics and dashboards are the next step (Chapter 13).

> **SHIFT-LEFT TESTING**
>
> A defect costs more to fix the later it is found. In this pipeline, a wrong GPA rounding rule is caught by a unit test in under a second on the developer's machine; the same bug found in production would mean incorrect transcripts issued to students.

4. Version control and Git workflows

4.1 Distributed version control

A version control system records every change to every file, so any earlier state can be inspected or restored and several people can work at once without overwriting each other. Git is *distributed*: every clone holds the complete history, so committing, branching, diffing and viewing the log all work offline. Each commit is identified by a SHA-1 hash of its content and its parent commits, which makes silent edits to history detectable.

4.2 The four areas a change passes through

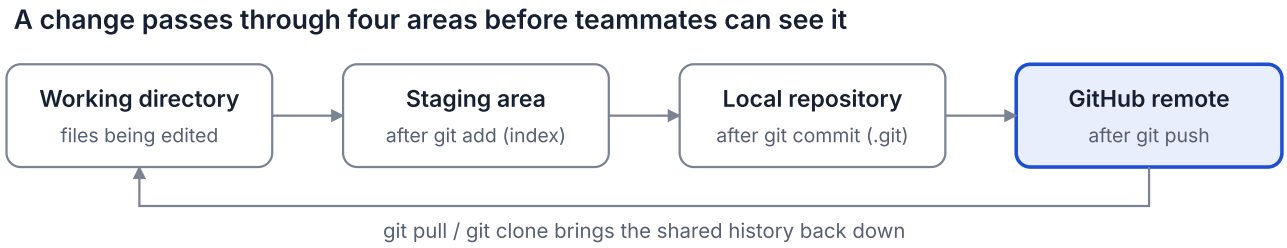


Figure 2. Working directory, staging area, local repository and GitHub remote

- **Working directory** — the checked-out files being edited.
- **Staging area (index)** — `git add` selects exactly which changes go into the next commit.
- **Local repository** — `git commit` records an immutable snapshot in the `.git` object database.
- **Remote repository** — `git push` publishes commits to GitHub, the team's shared source of truth; `git pull` brings others' work down.

4.3 Git versus GitHub

Aspect	Git	GitHub
What it is	Open-source version control engine (command line)	Hosted service for sharing and collaborating on Git repositories
Where it runs	On each developer's machine	In the cloud, through a web UI and REST API
Needs a network	No	Yes, to push, pull, review or trigger webhooks
Key features	Commits, branches, merges, diffs, tags	Pull requests, code review, issues, webhooks
Role here	Versions the application *and* its `Jenkinsfile`, `Dockerfile` and `k8s/` manifests	Holds the shared repository and pull-request history, and notifies Jenkins of new commits

4.4 Branching strategy used

The repository follows a feature-branch workflow. Figure 3 is drawn from its real history: 28 commits on six topic branches, each merged into `develop` with a no-fast-forward merge, then released to `main` and tagged `v1.0.0` (7 merge commits in total).

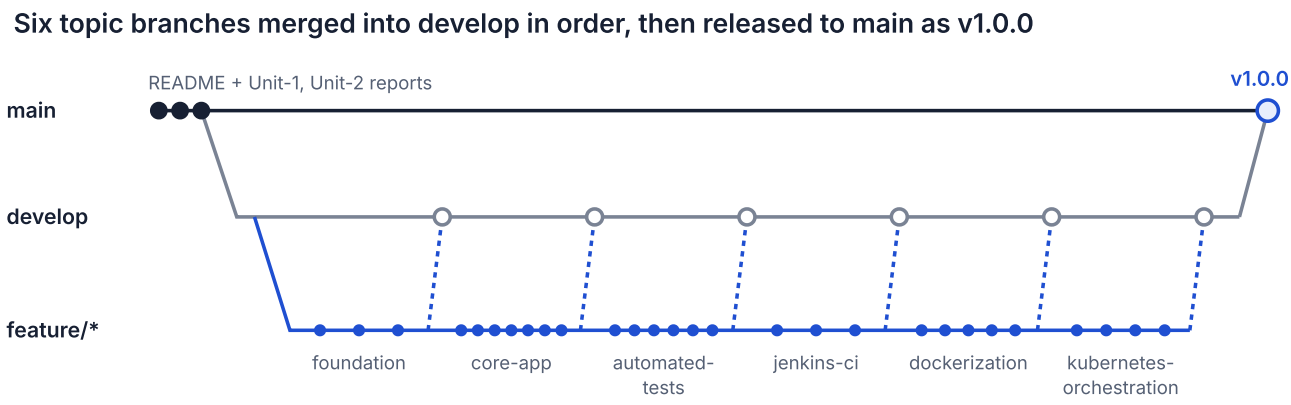


Figure 3. Branch history of this repository (dots are commits; dashed lines are merges into develop)

- `main` — production; receives only reviewed merges from `develop`, and Jenkins builds from it.
- `develop` — integration branch where finished topics come together.
- `feature/*` — one short-lived branch per topic: foundation, core-app, automated-tests, jenkins-ci, dockerization, kubernetes-orchestration.
- `hotfix/*` — reserved for urgent production fixes: cut from `main` and merged back into both `main` and `develop`. Not needed for this release.

5. Continuous integration with Jenkins

5.1 What continuous integration means

Continuous integration (CI) means merging small changes into the shared codebase often — ideally several times a day — and having each merge verified automatically by a build and test run. Problems are found while the change is still small and fresh in the author's mind, which removes the "integration hell" of long-lived branches. For this project the CI contract is simple: nothing reaches `main` unless lint, tests, the coverage gate and a container smoke test all pass.

5.2 The pipeline stages

#	Stage	What runs	Stops the build when
1	SCM Checkout	Clones the exact commit that triggered the build	The repository cannot be reached
2	Environment Setup & Dependencies	New virtualenv; `pip install -r requirements-dev.txt`	A dependency cannot be installed
3	Code Linting & Syntax Gate	flake8 on `app/`, `database/`, `tests/` and scripts	Any style, syntax or complexity finding
4	Automated Unit & Integration Tests	pytest with JUnit XML, coverage XML and HTML reports	A failing test, or coverage below 85%
5	Build Docker Artifact	`docker build` tagged `v1.0.<BUILD_NUMBER>` and labelled with the commit	The image fails to build
6	Container Smoke Test	Runs the new image and probes `/api/health` inside it	The app is not healthy within about 30 s
7	Kubernetes Minikube Deployment	`minikube image load`, apply manifests with the build's tag, wait for rollout	Rollout not ready in 180 s (marked UNSTABLE if Minikube is not running)

Reports are archived with every build, and the JUnit results feed Jenkins' test-trend view.

5.3 Jenkins architecture: controller and agents

Jenkins splits coordination from execution. The **controller** serves the web UI, stores job configuration and build history, receives triggers and schedules work; **agents** are the machines or containers that actually run the steps. Scaling out means adding agents, not making the controller bigger. This project runs on a single machine, where the built-in node acts as both.

Component	Responsibility in this project
GitHub + Poll SCM / webhook	Signals Jenkins that `main` has new commits
Jenkins controller	Schedules the pipeline, keeps console logs, test trends and archived reports
Agent (built-in node)	Runs Python, flake8, pytest, `docker`, `minikube` and `kubectl`
Plugins	Pipeline, Git, JUnit, Timestamper, Workspace Cleanup (all in “suggested plugins”)
Pipeline as code	The `Jenkinsfile` lives in the repository, so pipeline changes go through the same branches and pull requests as code

6. Containerization and Docker

6.1 Virtual machines versus containers

A virtual machine emulates a whole computer: a hypervisor runs a complete guest operating system, with its own kernel, for every application. That isolation is strong but heavy — gigabytes of disk and memory per VM, and boot times measured in minutes. A container instead shares the host's Linux kernel and isolates only the application's processes, using two kernel features:

- **Namespaces** give each container its own view of process IDs, network interfaces, mount points, hostname and users, so it cannot see or touch other containers.
- **Control groups (cgroups)** limit how much CPU, memory and I/O a container may use; this project's pods are capped at 250 millicores of CPU and 256 MiB of memory.

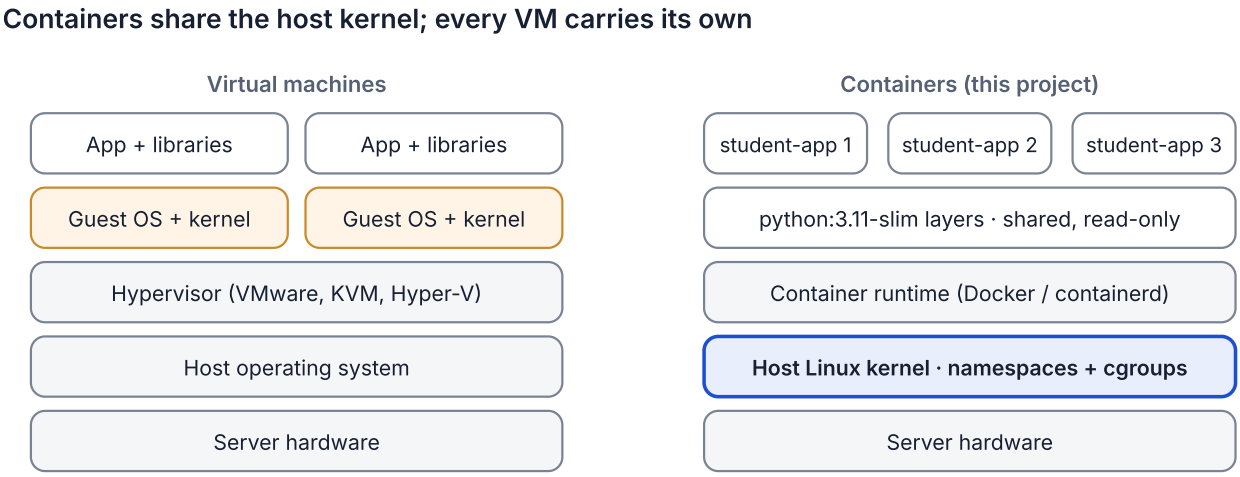


Figure 4. Virtual machines duplicate the operating system; the three app containers share one kernel

Attribute	Virtual machine	Container
Isolation level	Hardware, via a hypervisor	Operating system, via namespaces and cgroups
Kernel	One guest kernel per VM	Shared host kernel
Start-up time	Minutes (full OS boot)	Seconds or less (a process start)
Size	Gigabytes per image	Tens to hundreds of megabytes; this image is 62.8 MB compressed
Density	Tens per server	Hundreds per server
Portability	Tied to hypervisor formats	Standard OCI images run on any compliant runtime

6.2 Image layers and immutability

An image is a read-only stack of layers, one per Dockerfile instruction. Docker reuses a cached layer when neither the instruction nor its input files have changed, which is why this project's Dockerfile copies `requirements.txt` and installs dependencies *before* copying the source code: an ordinary code change rebuilds only the top layers (Figure 5).

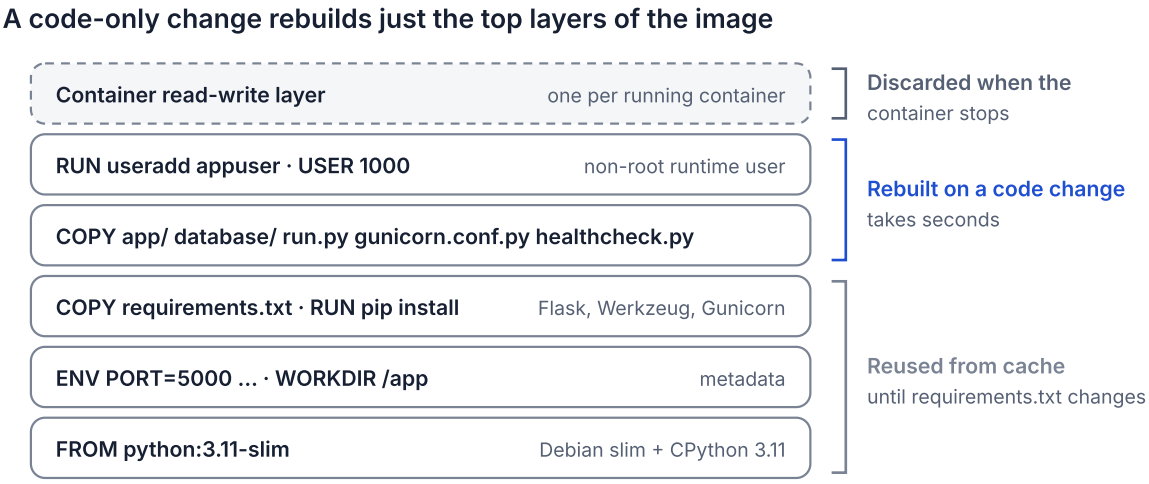


Figure 5. Layers of `student-academic-platform`, from the base image up to a running container

> **WHY IT MATTERS**
>
> Images are immutable: the exact image that passes the smoke test in Jenkins is the one Minikube runs. Nothing is installed or patched by hand on the way, which removes environment drift.

7. Kubernetes and Minikube orchestration

7.1 Why orchestration is needed

Docker can run a container, but on its own it will not keep three copies alive, replace one that crashes, spread traffic across them, or roll out a new version gradually. An orchestrator does this by comparing the *desired* state written in YAML with the *actual* state of the cluster and continuously correcting the difference. Each capability was demonstrated on this project's deployment (Chapter 11):

- **Self-healing** — a deleted pod was replaced and the Deployment was back to 3/3 ready in 9 seconds.

- **Load balancing and service discovery** — 12 requests to the stable name `student-service` were served by all three pods (7 / 4 / 1).
- **Zero-downtime rolling updates** — 188 health requests sent during a version change all succeeded.
- **Fast rollback** — `kubectl rollout undo` restored the previous version in one command.

7.2 Architecture and the objects used

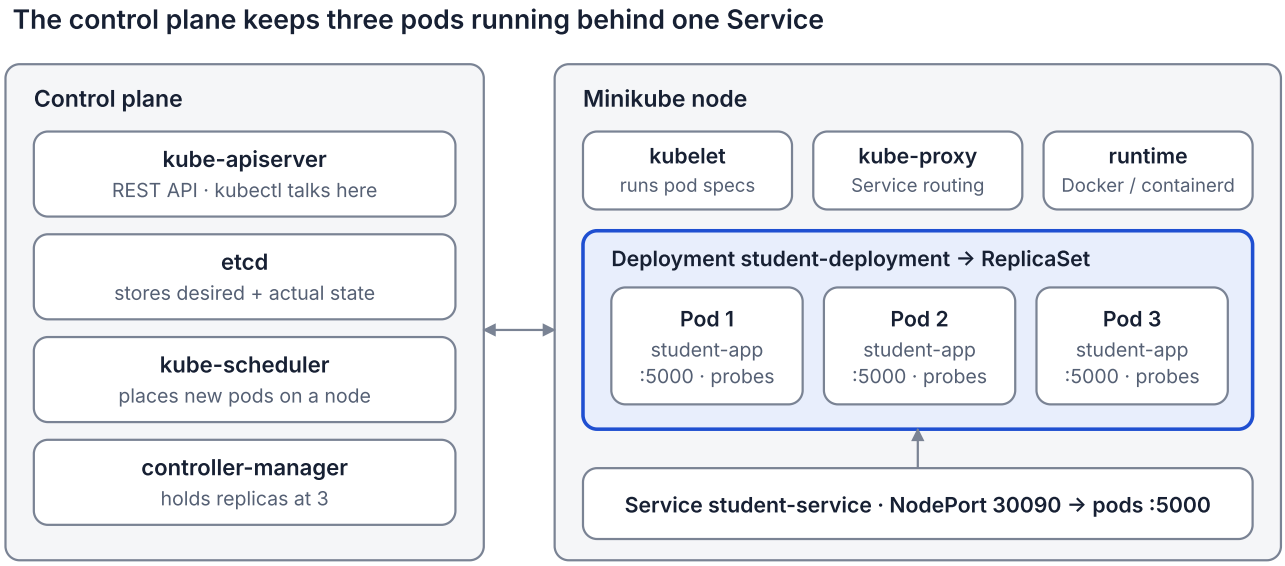


Figure 6. Control plane components and the objects running on the Minikube node

The **control plane** holds the cluster's brain: `kube-apiserver` is the single entry point for `kubectl` and every component; `etcd` stores the desired and actual state; `kube-scheduler` chooses a node for each new pod; and `kube-controller-manager` runs the loops that keep reality matching the specification. On each **node**, the `kubelet` starts the containers a pod needs through the container runtime, and `kube-proxy` programs the network rules that route Service traffic to healthy pods. Minikube packs all of this into a single local node.

Object	How this project uses it
Deployment	`student-deployment`: 3 replicas, rolling updates with `maxSurge: 1` and `maxUnavailable: 0`, 5 revisions kept for rollback
ReplicaSet	Created by the Deployment for each version; keeps exactly 3 pods of that version alive
Pod	One `student-app` container on port 5000 with readiness and liveness probes on `/api/health`, CPU/memory requests and limits, non-root user and read-only root filesystem
Service	`student-service`, type NodePort: a stable cluster IP and DNS name, exposed on node port 30090

8. The integrated DevOps ecosystem

Git, Jenkins, Docker and Kubernetes become one delivery system when each hands its output to the next automatically. Figure 7 shows that chain for this project.

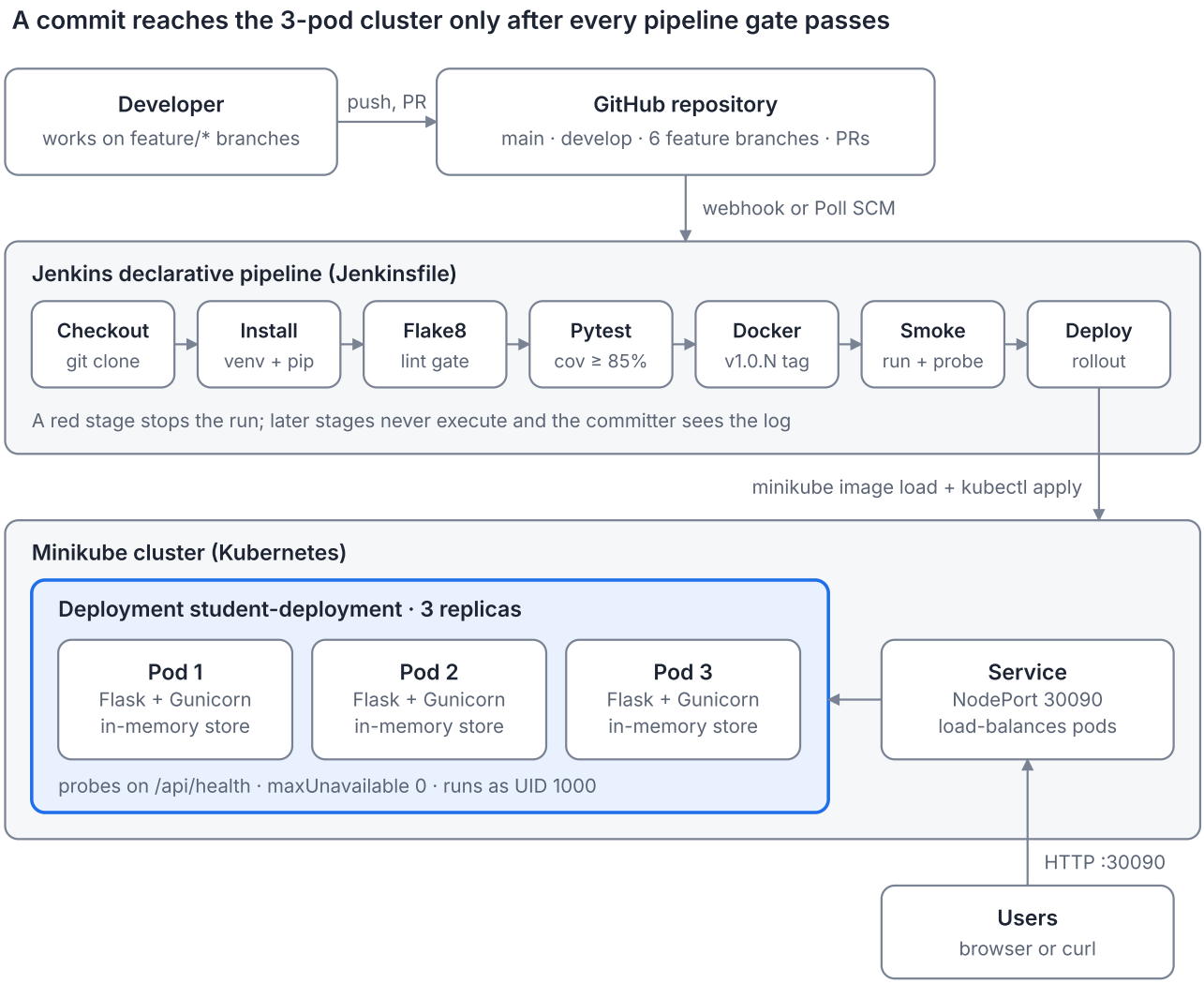


Figure 7. End-to-end flow: GitHub → Jenkins (7 stages) → Minikube Deployment behind a NodePort Service

8.1 One change, from commit to production

1. The developer creates a `feature/*` branch, commits and pushes it to GitHub, and opens a pull request into `develop`.
2. After review the pull request is merged; a release pull request then promotes `develop` to `main`.
3. Jenkins detects the new commit on `main` (webhook or Poll SCM) and checks it out.
4. It installs dependencies, runs flake8 and the 107 tests, and enforces the 85% coverage gate.
5. It builds `student-academic-platform:v1.0.<N>` and smoke-tests the container.
6. It loads the image into Minikube and applies the manifests with that exact tag.
7. Kubernetes starts new pods, waits for their readiness probes, shifts traffic to them and removes the old ones.
8. Health checks and logs confirm the release; if anything is wrong, `kubectl rollout undo` brings back the previous version.

8.2 Trainer case study versus this project

Component	Trainer case study (Library)	This project (Academic platform)
Domain	Library catalogue and circulation	Student records and academic grading
Entities	Users, Books, Loans	Students, Faculty, Courses, Enrollments, Grades
Core business rule	Max 3 books per student	Max 18 credits per student per semester
Test focus	Stock deduction, returns	GPA calculation, enrollment conflict checks
Test suites	`test_book_service.py`, `test_borrow_return.py`	`test_student_service.py`, `test_enrollment_service.py`, `test_marks_mgmt.py`, `test_models.py`, `test_api_endpoints.py`
Image	`library-management-platform:v1.0.0`	`student-academic-platform:v1.0.0`
Kubernetes	`library-deployment`, NodePort 30080	`student-deployment`, `student-service`, NodePort 30090

Part B — Implementation and verification

Chapters 9–14 describe what was built, how to reproduce it, the evidence that it works, and what was learnt along the way.

9. Application design

The platform enforces these rules in `app/services/academic_service.py`; each has its own automated tests.

- Unique student IDs and emails, faculty IDs and course codes; only `ACTIVE` students may enrol.
- No duplicate enrolment, no timetable clash (two courses in the same slot and term), and course capacity is never exceeded.
- Prerequisites must have been **passed** — an F does not count.
- At most **18 credits** per student per semester.
- A graded course cannot be dropped; a student with active enrolments cannot be deleted.

Grades use a 10-point scale. SGPA is the credit-weighted average for one term and CGPA the same over all terms, rounded half-up to two decimals; a failed course counts towards credits attempted but not credits earned.

Marks	90–100	80–89	70–79	60–69	50–59	45–49	40–44	< 40
Grade	O	A+	A	B+	B	C	P	F
Grade points	10	9	8	7	6	5	4	0

$$\text{SGPA} = \frac{\Sigma\ (\text{credits} \times \text{grade points})}{\Sigma\ \text{credits}}$$

Inside each pod the code is layered: `routes/` (HTTP) → `services/` (academic rules and grading) → `models/` (validation) → `database/` (thread-safe in-memory store). Domain errors carry their own HTTP status (400, 404, 409, 422), so the API layer stays thin.

Method	Path	Purpose
GET	`/api/health`	Liveness/readiness probe; includes the serving pod name
GET / POST	`/api/students`	List (`?q=` search) / register a student
GET / DELETE	`/api/students/<id>`	Fetch / delete a student
GET	`/api/students/<id>/transcript`	Grades, SGPA per term, CGPA, credits earned
GET / POST	`/api/faculty`, `/api/courses`	List / add faculty and courses (`?term=` filter)
POST	`/api/enrollments`, `/api/enrollments/drop`	Enrol in or drop a course
POST	`/api/grades`	Record marks; grade and points are derived
GET	`/`	HTML dashboard

10. Setup steps

10.1 Workstation

macOS (Homebrew) — Jenkins runs as the logged-in user, so it shares that user's Docker Desktop and Minikube:

```
brew install git kubectl minikube jenkins-lts
brew install --cask docker                # Docker Desktop; open it once so the engine starts
brew services start jenkins-lts           # http://localhost:8080
cat ~/.jenkins/secrets/initialAdminPassword
minikube start --driver=docker
```

Ubuntu 22.04/24.04 or WSL2 — `./scripts/setup_ubuntu.sh` installs Python, Git, Docker, kubectl, Minikube, OpenJDK 21 and Jenkins, and adds the `jenkins` user to the `docker` group. In both cases, unlock Jenkins with the printed password and choose **Install suggested plugins**.

10.2 Repository and branches (Topic 1)

```
cd DEVOPS-1/Unit-3                                  # the project's folder in the course repository
git log --graph --oneline --all                     # six topic branches, merged in order
bash scripts/publish_to_github.sh amaan974/DEVOPS-1   # push + one merged PR per topic
```

10.3 Application and tests (Topics 2–3)

```
cd DEVOPS-1/Unit-3
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt        # runtime + pytest, pytest-cov, flake8
PORT=5001 python run.py                    # http://localhost:5001 (5000 is AirPlay on macOS)
pytest -v --cov=app --cov=database --cov-fail-under=85 --junitxml=reports/test-results.xml
```

10.4 Jenkins pipeline (Topic 4)

1. New Item → Pipeline → *Pipeline script from SCM* → Git, `https://github.com/amaan974/DEVOPS-1.git`, branch `*/main`, script path `Unit-3/Jenkinsfile`.
2. Trigger with **Poll SCM** (`H/5 * * * *`) or a GitHub webhook to `/github-webhook/`.
3. Build Now; the seven stages of Chapter 5.2 run and the reports are archived.

10.5 Docker (Topic 5)

```
docker build -t student-academic-platform:v1.0.0 .
docker images student-academic-platform
docker run -d -p 5001:5000 --name student-container student-academic-platform:v1.0.0
docker ps                                  # STATUS shows (healthy)
curl http://localhost:5001/api/health
```

10.6 Kubernetes on Minikube (Topic 6)

```
./scripts/deploy_minikube.sh                        # build inside Minikube, apply k8s/, wait
kubectl get pods -l app=student-platform -o wide
minikube service student-service --url              # open the dashboard
HOST_PORT=5001 ./scripts/capture_evidence.sh        # write evidence files
```

11. Verification results and evidence

All six checklist items are covered by the repository, the screenshots in this chapter and the command output in the Appendix.

Verification environment: Docker Engine 29.8.2 and a single-node Kubernetes v1.30.4 cluster (k3s, used as a Minikube-equivalent local cluster) on Ubuntu 24.04. Docker Hub was unreachable from that environment, so the image was built on a stand-in for `python:3.11-slim` (Debian 12 bookworm-slim + CPython 3.11.9) passed in through the Dockerfile's `BASE_IMAGE` build argument; the Dockerfile default remains `python:3.11-slim`. Lint and the full test suite were also run on Python 3.14.2 (the version on the development Mac) with identical results.

11.1 Automated tests

Suite	Tests	What it proves
`test_marks_mgmt.py`	32	Grade boundaries (89.99 = A+, 90 = O), invalid marks, SGPA/CGPA, half-up rounding, failed courses, transcripts
`test_models.py`	24	Field validation and normalisation for students, faculty and courses
`test_enrollment_service.py`	20	Duplicate enrolment, prerequisites, timetable clash, 18-credit limit (17 + 2 rejected, exactly 18 allowed), capacity, drop rules
`test_api_endpoints.py`	17	Every endpoint; status codes 200/201/400/404/405/409/422; JSON error bodies
`test_student_service.py`	14	Student CRUD, duplicate ID and email, delete protection
Total	**107**	99.71% coverage; flake8 0 findings

11.2 Docker and Kubernetes behaviour

Check	Command	Observed
Container health	`docker ps`	`Up (healthy)`; process runs as `uid=1000(appuser)`
Replicas	`kubectl get pods -l app=student-platform`	3 pods `1/1 Running`
Self-healing	delete one pod, poll the Deployment	Replacement scheduled within 1 s; 3/3 ready after 9 s
Load balancing	12 requests to `http://student-service:5000` from inside the cluster	Served by all 3 pods (7 / 4 / 1)
NodePort	`curl http://<node-ip>:30090/api/health`	`"status":"UP"` with the serving pod's name
Rolling update	deploy `v1.0.7` with `maxSurge: 1`, `maxUnavailable: 0`	188 of 188 health requests succeeded during the rollout
Rollback	`kubectl rollout undo deployment/student-deployment`	Back to `v1.0.0`, 3/3 ready

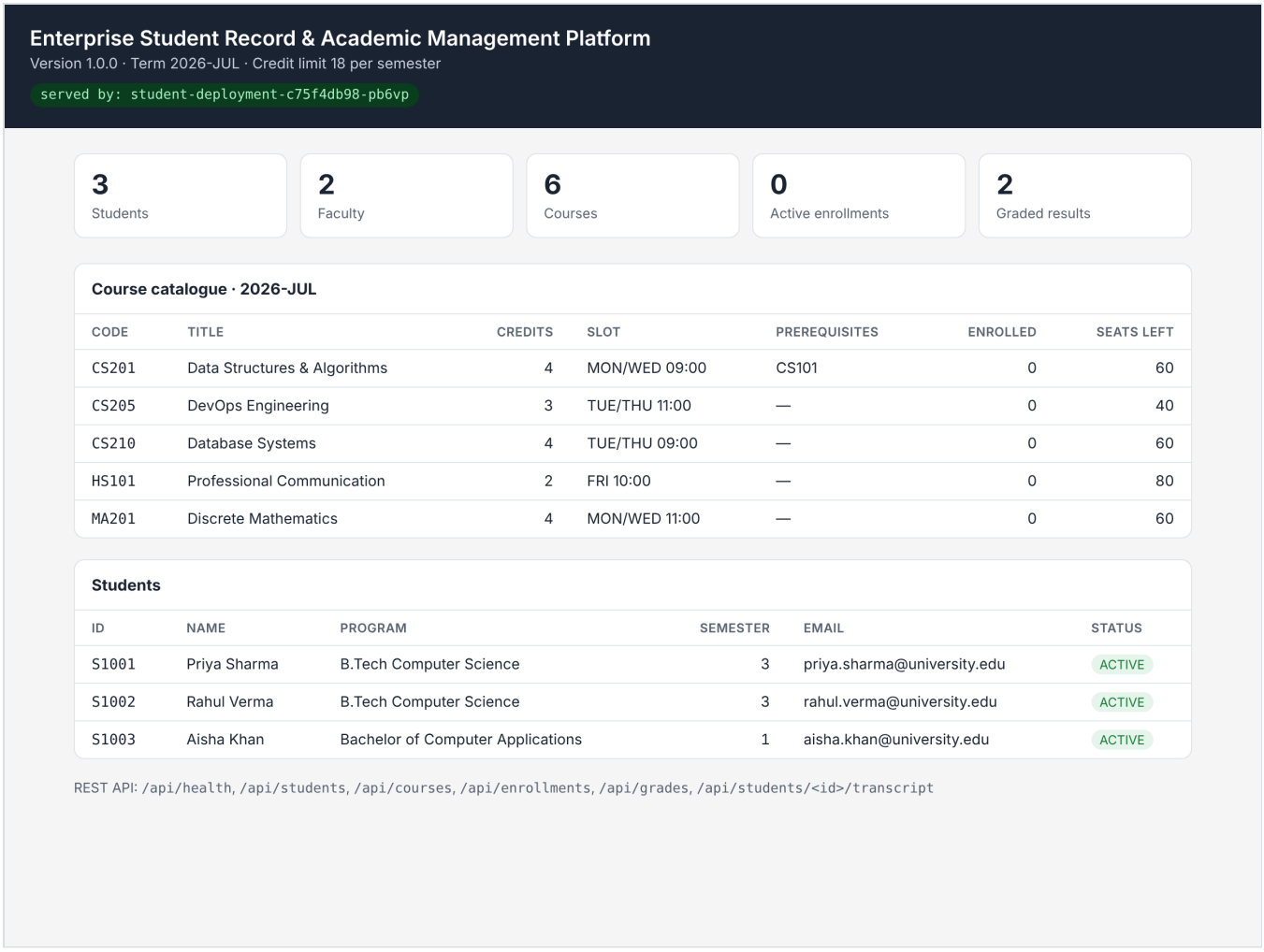


Figure 8. Dashboard served through NodePort 30090; the green badge names the pod that answered

11.3 Evidence index

Checklist item	Evidence
1. GitHub repository	Repository with `app/`, `database/`, `tests/`, `k8s/`, `Dockerfile`, `Jenkinsfile` in the `Unit-3` folder of https://github.com/amaan974/DEVOPS-1
2. Branching and PR evidence	Figure 3 and Appendix A.1: 28 commits and 7 merges across 6 topic branches
3. Pipeline run and JUnit test report	Section 11.4: pipeline output from checkout to smoke test, JUnit report (107 tests, 0 failures) and coverage report
4. Docker build artifacts	Appendix A.3: `docker images` and `docker ps` showing `(healthy)`
5. Minikube orchestration	Appendix A.4–A.5 and Section 11.2: 3 healthy replicas, self-healing, load balancing, rolling update, rollback
6. Technical report	This document

11.4 Pipeline run, test report and coverage

The pipeline's stages were run end to end with `scripts/run_ci_locally.sh`, which executes the same commands as the `Jenkinsfile`. The screenshots below come from that run.

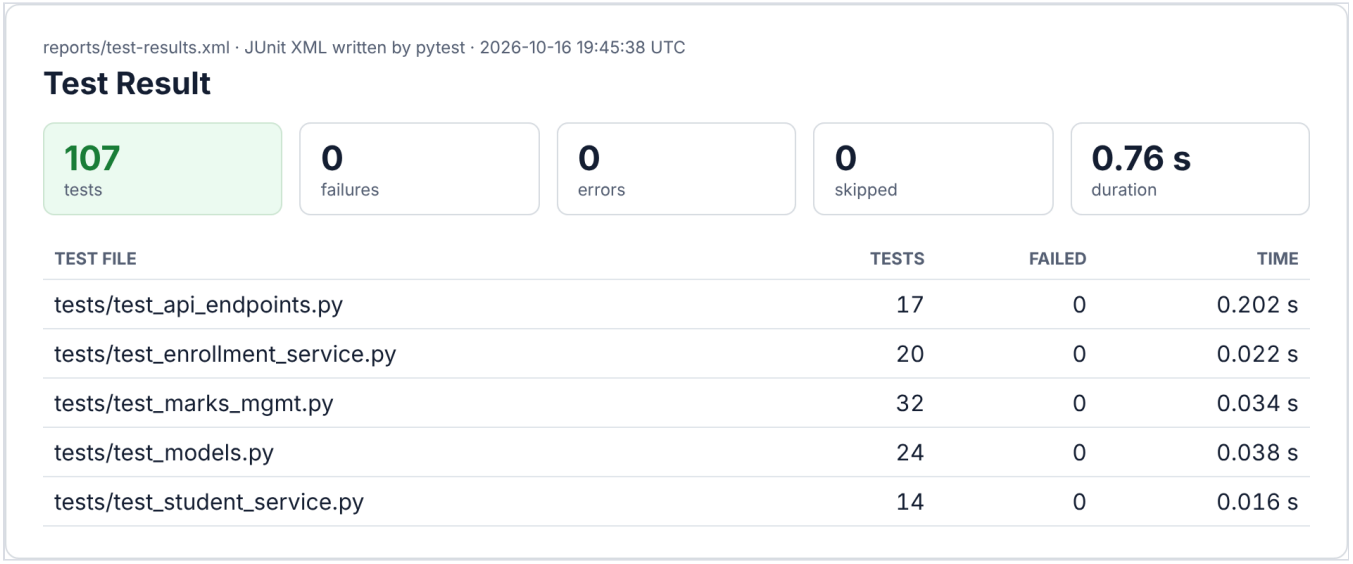
```
~/DEVOPS-1/Unit-3 — ./scripts/run_ci_locally.sh

==================== [Stage] SCM Checkout ====================
Commit 5de57b6 by Mohammed Amaan: Release v1.0.0: merge develop into main
================== [Stage] Environment Setup & Dependencies ==================
  … dependencies installed (pip freeze, 15 lines) omitted …
==================== [Stage] Code Linting & Syntax Gate ====================
0
flake8: no issues found
==================== [Stage] Automated Unit & Integration Tests ====================
  … 107 individual PASSED lines and the coverage table omitted …
Required test coverage of 85% reached. Total coverage: 99.71%
============================== 107 passed in 0.75s ==============================
==================== [Stage] Build Docker Artifact ====================
  … docker build steps #1 to #12 omitted …
#13 naming to docker.io/library/student-academic-platform:v1.0.1 done
#13 naming to docker.io/library/student-academic-platform:latest done
IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
student-academic-platform:latest   aca2b36cc6bc        267MB         62.8MB
student-academic-platform:v1.0.0   32ca6a294584        267MB         62.8MB   U
student-academic-platform:v1.0.1   aca2b36cc6bc        267MB         62.8MB
student-academic-platform:v1.0.7   797a73d53e0f        267MB         62.8MB

==================== [Stage] Container Smoke Test ====================
efd970d485fe600054236435529ddd55f810201cf0554cc827f1d54e0ab00db0
HEALTHY: {"status": "UP", "service": "Enterprise Student Record & Academic Management Platform", "version": "1.0.1", "instance": "e
fd970d485fe"}
Smoke test passed on attempt 1

LOCAL PIPELINE SUCCEEDED for student-academic-platform:v1.0.1
```

Figure 9. Pipeline output, stage by stage, from checkout to a passing container smoke test (repetitive lines omitted, as marked)



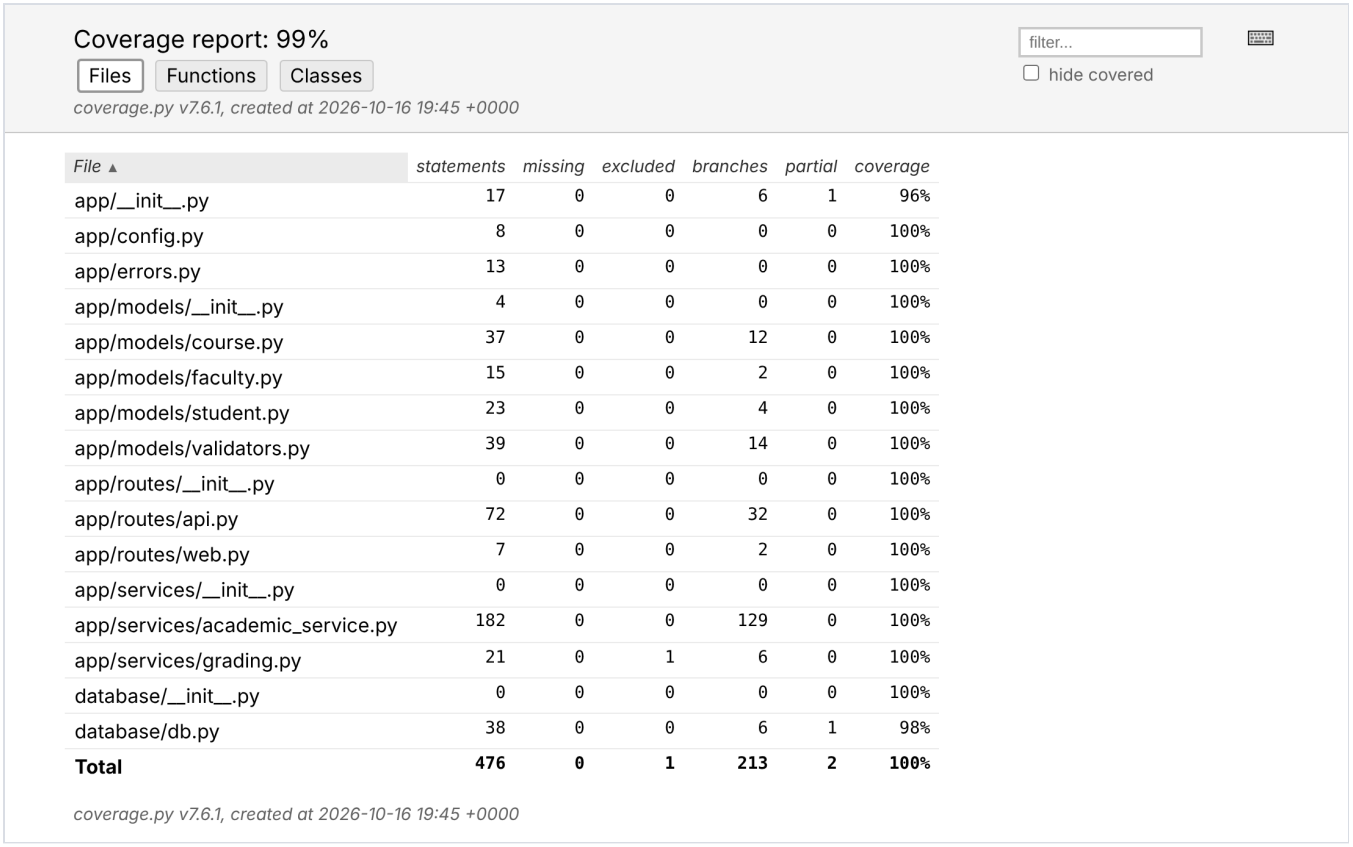
reports/test-results.xml · JUnit XML written by pytest · 2026-10-16 19:45:38 UTC

Test Result

107	0	0	0	0.76 s
tests	failures	errors	skipped	duration

TEST FILE	TESTS	FAILED	TIME
tests/test_api_endpoints.py	17	0	0.202 s
tests/test_enrollment_service.py	20	0	0.022 s
tests/test_marks_mgmt.py	32	0	0.034 s
tests/test_models.py	24	0	0.038 s
tests/test_student_service.py	14	0	0.016 s

Figure 10. JUnit XML test report produced by the run: 107 tests, 0 failures, 0 errors



Coverage report: 99%

Files | Functions | Classes

filter... hide covered

coverage.py v7.6.1, created at 2026-10-16 19:45 +0000

File ▲	statements	missing	excluded	branches	partial	coverage
app/__init__.py	17	0	0	6	1	96%
app/config.py	8	0	0	0	0	100%
app/errors.py	13	0	0	0	0	100%
app/models/__init__.py	4	0	0	0	0	100%
app/models/course.py	37	0	0	12	0	100%
app/models/faculty.py	15	0	0	2	0	100%
app/models/student.py	23	0	0	4	0	100%
app/models/validators.py	39	0	0	14	0	100%
app/routes/__init__.py	0	0	0	0	0	100%
app/routes/api.py	72	0	0	32	0	100%
app/routes/web.py	7	0	0	2	0	100%
app/services/__init__.py	0	0	0	0	0	100%
app/services/academic_service.py	182	0	0	129	0	100%
app/services/grading.py	21	0	1	6	0	100%
database/__init__.py	0	0	0	0	0	100%
database/db.py	38	0	0	6	1	98%
Total	**476**	**0**	**1**	**213**	**2**	**100%**

coverage.py v7.6.1, created at 2026-10-16 19:45 +0000

Figure 11. HTML coverage report produced by the run: 99.71% of statements and branches covered

12. Challenges and resolutions

9 problems surfaced while adapting the case study. Most came from the seams between tools — a pipeline that builds an image versus a cluster that must run it — and two were specific to macOS.

#	Challenge	Root cause	Resolution
1	Pipeline built `v1.<N>` but the cluster kept running `v1.0.0`	The case-study Jenkinsfile tags images by build number while the manifest hard-codes `v1.0.0`	The deploy stage renders the manifest with the build's own tag (`sed … | kubectl apply -f -`) and records a change-cause, so `rollout history` names each build
2	Pods would sit in `ErrImageNeverPull` on Minikube	Images built by the host Docker daemon are not inside the Minikube node	The pipeline runs `minikube image load`; the manual path builds inside `minikube docker-env`
3	Data would differ between requests in one pod	The store lives in process memory; several Gunicorn worker processes would each hold a copy	Gunicorn runs 1 worker × 4 threads; mutating operations hold a re-entrant lock so seat and credit checks are atomic
4	Pods refused under `runAsNonRoot`	Kubernetes cannot verify that a named user (`USER appuser`) is non-root	`USER 1000` in the Dockerfile plus `runAsUser: 1000` in the pod security context
5	Known vulnerabilities in the case study's pinned versions	Werkzeug 3.0.3 and Gunicorn 22.0.0 predate security fixes	Pinned Werkzeug 3.0.6 (CVE-2024-49767, CVE-2024-49766) and Gunicorn 23.0.0 (CVE-2024-6827) [2][3][4]
6	Jenkins apt install fails with the manual's signing key	Jenkins rotated its repository signing key in December 2025 [1]	The Ubuntu setup script uses `jenkins.io-2026.key` and Java 21

7	A GPA of 9.125 rounded to 9.12	Python's `round()` works on binary floats	`Decimal` with `ROUND_HALF_UP`; a test pins 9.125 → 9.13
8	Flask and the container could not bind port 5000 on macOS	macOS AirPlay Receiver listens on port 5000	`run.py` and Gunicorn read `PORT`; run with `PORT=5001`, publish as `-p 5001:5000`; `capture_evidence.sh` takes `HOST_PORT`
9	Jenkins on macOS could not find `docker`, `minikube` or Homebrew's `python3`	A Jenkins service started by Homebrew does not inherit the shell's `PATH`	The Jenkinsfile prepends `/opt/homebrew/bin:/usr/local/bin` to `PATH`; harmless on Linux

13. Reflections and next steps

- **State versus replicas.** Each pod keeps its own in-memory data, so an enrolment made through one pod is invisible to the other two. A real registrar would make the pods stateless and keep state in PostgreSQL, with schema migrations run as a Kubernetes Job before each rollout.
- **Registry instead of image loading.** `minikube image load` suits one local node; a shared cluster would push each tagged image to a registry (Docker Hub, Amazon ECR or Harbor) and let nodes pull it.
- **Security gates.** The image already runs as non-root with a read-only root filesystem and no Linux capabilities. An image scan (for example Trivy) and a dependency audit would be the next pipeline stages.
- **Observability.** Health probes show liveness only. A `/metrics` endpoint scraped by Prometheus, Grafana dashboards and structured logs would close the Monitor phase and make MTTR measurable.
- **Elasticity and governance.** With requests and limits already set, a HorizontalPodAutoscaler could scale from 3 to 10 pods at registration time; branch protection on `main` with a required green Jenkins check would make the gates mandatory.

14. Conclusion

The project meets every Unit 3 deliverable: versioned source on topic branches, a seven-stage Jenkins pipeline with an 85% coverage gate, a hardened container image, and a self-healing three-replica Kubernetes deployment that updates without downtime. Most of the learning sat at the seams between tools — image tags, where an image lives, which user runs what, and where state is kept — rather than inside any single tool.

References

1. Jenkins project, “Linux repository signing keys changing”, 23 Dec 2025. https://www.jenkins.io/blog/2025/12/23/repository-signing-keys-changing/
2. Debian security tracker, CVE-2024-6827 (Gunicorn request smuggling; fixed in 23.0.0). https://security-tracker.debian.org/tracker/CVE-2024-6827
3. NIST NVD, CVE-2024-49767 (Werkzeug multipart memory exhaustion; fixed in 3.0.6). https://nvd.nist.gov/vuln/detail/CVE-2024-49767
4. NIST NVD, CVE-2024-49766 (Werkzeug safe_join on Windows; fixed in 3.0.6). https://nvd.nist.gov/vuln/detail/CVE-2024-49766
5. Course material: Unit 3 Document 1 (theoretical and architectural reference) and Document 2 (Library platform implementation guide).

Appendix. Key command output

Excerpts from the verification run described in Chapter 11; lines removed for length are marked.

A.1 Git branch history

```
# Captured 2026-10-16 19:45:33 UTC on host vm

$ git log --graph --oneline --decorate --all -n 50
*   5de57b6 (HEAD -> main, tag: v1.0.0) Release v1.0.0: merge develop into main
|\
| *   3c3052b (develop) Merge branch 'feature/kubernetes-orchestration' into develop
| |\
| | * c28f65f (feature/kubernetes-orchestration) Document Minikube deployment, self-healing and submission evidence
| | * 65373b6 Add Minikube deploy, evidence capture and GitHub publishing scripts
| | * b431d93 CI: load image into Minikube and roll out the build's exact tag
| | * 2dc6044 Add Kubernetes Deployment (3 replicas, probes, zero-downtime rolling update) and NodePort 30090 Service
| * | bf28b20 Merge branch 'feature/dockerization' into develop
| |\|
| | * 5b72ccc (feature/dockerization) Document Docker build and run
| | * c32cbb0 CI: build versioned Docker image and smoke-test the container
| | * 277c500 Add production Dockerfile (cached deps layer, non-root, HEALTHCHECK) and .dockerignore
| | * 3bc36dc Add standard-library health probe for container health checks
| | * a230314 Serve with Gunicorn: one worker, four threads
| * | 03e3625 Merge branch 'feature/jenkins-ci' into develop
| |\|
| | * 8913da7 (feature/jenkins-ci) Document Jenkins installation and pipeline job setup
| | * 24b3bf4 Add workstation setup script and local CI runner
| | * 9afdf9f Add declarative Jenkins pipeline: checkout, dependencies, lint, tests with JUnit report
| * | 35c4414 Merge branch 'feature/automated-tests' into develop
| |\|
| | * 59b98e3 (feature/automated-tests) Document the test suites and coverage gate
| | * cea92cb Add HTTP integration tests for every API endpoint
| | * 896ce71 Add marks management tests: grade boundaries, SGPA/CGPA, failed courses
| | * abcd2d6 Add enrollment rule tests: duplicates, prerequisites, clashes, credit limit, capacity
| | * 9fec313 Add unit tests for entity validation and student service
| | * e170be2 Add pytest and coverage configuration and dev requirements
| * | abbb8d2 Merge branch 'feature/core-app' into develop
| |\|
| | * e6bd7b5 (feature/core-app) Document the REST API
| | * 80e0846 Add HTML dashboard showing catalogue, students and serving instance
| | * e90948a Expose REST API for students, faculty, courses, enrollments and grades
| | * 924e019 Implement academic service: 18-credit limit, prerequisites, timetable clashes, capacity, marks and
transcripts
| | * a7c2a3a Implement 10-point grading scale and credit-weighted SGPA/CGPA
| | * bedc910 Add domain errors and validated Student, Faculty and Course models
| | * 6dd545f Add thread-safe in-memory database with seed data
| * | 896caa3 Merge branch 'feature/foundation' into develop
|/| |
| |/
| * cf5dbc6 (feature/foundation) Document scope, business rules, repository layout and branching strategy
| * c679695 Add Flask application factory, config and /api/health endpoint
| * ce3ee2a Add pinned runtime requirements (Flask 3.0, Werkzeug 3.0) and flake8 style config
|/
* b0a7a5b Add Unit 2 mini project: Agile (Scrum) report
* 71a1830 Add Unit 1 mini project: Design Thinking & product planning report
* e50950c Initial commit: course repository README and Unit-3 .gitignore

$ git branch -a
  develop
  feature/automated-tests
  feature/core-app
  feature/dockerization
  feature/foundation
  feature/jenkins-ci
  feature/kubernetes-orchestration
* main
```

A.2 Lint, tests and coverage

```
==================== [Stage] Code Linting & Syntax Gate ====================
0
```

```
flake8: no issues found

(... 107 individual 'PASSED' test lines omitted ...)

---------- coverage: platform linux, python 3.13.16-final-0 ----------
Name                              Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------
app/__init__.py                      17      0      6      1    96%   9->12
app/config.py                         8      0      0      0   100%
app/errors.py                        13      0      0      0   100%
app/models/__init__.py                4      0      0      0   100%
app/models/course.py                 37      0     12      0   100%
app/models/faculty.py                15      0      2      0   100%
app/models/student.py                23      0      4      0   100%
app/models/validators.py             39      0     14      0   100%
app/routes/__init__.py                0      0      0      0   100%
app/routes/api.py                    72      0     32      0   100%
app/routes/web.py                     7      0      2      0   100%
app/services/__init__.py              0      0      0      0   100%
app/services/academic_service.py    182      0    129      0   100%
app/services/grading.py              21      0      6      0   100%
database/__init__.py                  0      0      0      0   100%
database/db.py                       38      0      6      1    98%   122->121
-----------------------------------------------------------------------------
TOTAL                               476      0    213      2    99%
Coverage HTML written to dir reports/htmlcov
Coverage XML written to file reports/coverage.xml

Required test coverage of 85% reached. Total coverage: 99.71%
============================== 107 passed in 0.75s ==============================
```

A.3 Docker image and running container

```
# Captured 2026-10-11 18:06:20 UTC on host vm

$ docker images student-academic-platform
IMAGE                             ID             DISK USAGE   CONTENT SIZE   EXTRA
student-academic-platform:latest   06feef280a39        267MB         62.8MB
student-academic-platform:v1.0.0   32ca6a294584        267MB         62.8MB
student-academic-platform:v1.0.1   06feef280a39        267MB         62.8MB
student-academic-platform:v1.0.7   797a73d53e0f        267MB         62.8MB

$ docker run -d -p 5000:5000 --name student-container student-academic-platform:v1.0.0
ef9afec335c67557aab0f5162de9a428d2efabb28e3685249819463539898c2e

# waiting for the Docker HEALTHCHECK to report 'healthy'...

$ docker ps --filter name=student-container
CONTAINER ID   IMAGE                              COMMAND                  CREATED         STATUS
PORTS                    NAMES
ef9afec335c6   student-academic-platform:v1.0.0   "gunicorn --config g…"   6 seconds ago   Up 6 seconds (healthy)
0.0.0.0:5000->5000/tcp   student-container

$ curl -s --noproxy * http://localhost:5000/api/health
{"status":"UP","service":"Enterprise Student Record & Academic Management
Platform","version":"1.0.0","instance":"ef9afec335c6"}

$ docker exec student-container id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
```

A.4 Self-healing

```
# Captured 2026-10-11 18:06:27 UTC on host vm

## BEFORE: Deployment declares 3 replicas
$ k3s kubectl get pods -l app=student-platform -o wide
NAME                                  READY   STATUS    RESTARTS   AGE     IP           NODE   NOMINATED NODE
READINESS GATES
student-deployment-c75f4db98-pb6vp   1/1     Running   0          22m     10.42.0.37   vm     <none>           <none>
student-deployment-c75f4db98-rnhcq   1/1     Running   0          9m43s   10.42.0.40   vm     <none>           <none>
student-deployment-c75f4db98-w4kwk   1/1     Running   0          22m     10.42.0.36   vm     <none>           <none>

## Simulating a crash: deleting pod student-deployment-c75f4db98-pb6vp
```

```
$ k3s kubectl delete pod student-deployment-c75f4db98-pb6vp --wait=false
pod "student-deployment-c75f4db98-pb6vp" deleted

## IMMEDIATELY AFTER: the ReplicaSet has already scheduled a replacement
$ k3s kubectl get pods -l app=student-platform -o wide
NAME                                  READY   STATUS              RESTARTS   AGE     IP           NODE   NOMINATED NODE
READINESS GATES
student-deployment-c75f4db98-pb6vp    1/1     Terminating         0          22m     10.42.0.37   vm     <none>
<none>
student-deployment-c75f4db98-rnhcq    1/1     Running             0          9m45s   10.42.0.40   vm     <none>
<none>
student-deployment-c75f4db98-w4kwk    1/1     Running             0          22m     10.42.0.36   vm     <none>
<none>
student-deployment-c75f4db98-x5t46    0/1     ContainerCreating   0          1s      <none>       vm     <none>
<none>

## AFTER 9s: back to 3/3 ready replicas; student-deployment-c75f4db98-pb6vp is gone
$ k3s kubectl get pods -l app=student-platform -o wide
NAME                                  READY   STATUS    RESTARTS   AGE     IP           NODE   NOMINATED NODE
READINESS GATES
student-deployment-c75f4db98-rnhcq    1/1     Running   0          9m54s   10.42.0.40   vm     <none>           <none>
student-deployment-c75f4db98-w4kwk    1/1     Running   0          22m     10.42.0.36   vm     <none>           <none>
student-deployment-c75f4db98-x5t46    1/1     Running   0          10s     10.42.0.41   vm     <none>           <none>

$ k3s kubectl get deployment student-deployment
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
student-deployment   3/3     3            3           23m
```

A.5 Service and load balancing

```
# Captured 2026-10-11 18:06:39 UTC on host vm

$ k3s kubectl get svc student-service -o wide
NAME              TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE   SELECTOR
student-service   NodePort   10.43.34.238   <none>        5000:30090/TCP   23m   app=student-platform

$ k3s kubectl get endpoints student-service
NAME              ENDPOINTS                                          AGE
student-service   10.42.0.36:5000,10.42.0.40:5000,10.42.0.41:5000   23m

## 12 requests to the Service DNS name from inside the cluster (kube-proxy spreads them over the pods)
$ k3s kubectl exec deploy/student-deployment -- python -c <probe script>
request  1 -> served by student-deployment-c75f4db98-x5t46
request  2 -> served by student-deployment-c75f4db98-x5t46
request  3 -> served by student-deployment-c75f4db98-x5t46
request  4 -> served by student-deployment-c75f4db98-rnhcq
request  5 -> served by student-deployment-c75f4db98-x5t46
request  6 -> served by student-deployment-c75f4db98-rnhcq
request  7 -> served by student-deployment-c75f4db98-x5t46
request  8 -> served by student-deployment-c75f4db98-w4kwk
request  9 -> served by student-deployment-c75f4db98-rnhcq
request 10 -> served by student-deployment-c75f4db98-x5t46
request 11 -> served by student-deployment-c75f4db98-rnhcq
request 12 -> served by student-deployment-c75f4db98-x5t46
requests per pod: {'student-deployment-c75f4db98-x5t46': 7, 'student-deployment-c75f4db98-rnhcq': 4, 'student-
deployment-c75f4db98-w4kwk': 1}

## From outside the cluster through NodePort 30090
$ curl -s -m 5 --noproxy * http://192.0.2.2:30090/api/health
{"status":"UP","service":"Enterprise Student Record & Academic Management
Platform","version":"1.0.0","instance":"student-deployment-c75f4db98-w4kwk"}
```